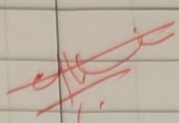
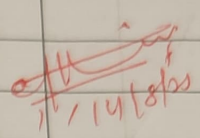
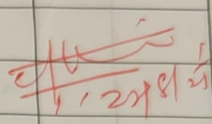
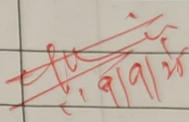


INDEX

NAME: Nirangan Gabe. S

ROLL NO.: 008

STD.: _____ DIV./SEC.: _____ SUBJECT: _____

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.	24.04.25	Exploring The Deep Learning Platforms.		
2.	7.08.25	Implement a classifier using open-source Dataset.		
3.	7.08.2025	Study of the classifier with respect to statistical parameters.		
4.	14.08.2025	Build a single Feed Forward neural network to recognise handwritten characters.		
5.	22.08.25	Study of activation function and its role.		
6.	09.9.25	Implement gradient descent & backpropagation in deep neural network.		

14.08.25

Exp 4: BUILD A SINGLE FEED FORWARD NEURAL NETWORK TO RECOGNISE HANDWRITTEN CHARACTERS.

AIM:

To Build a Single feed forward neural network to ~~seg~~ recognise handwritten characters.

ALGORITHM:

- * Start.

- * Import the MNIST dataset.

- * Preprocess Data :

 - Normalize pixel values ($0-255 \rightarrow 0-1$).

 - Flatten 28×28 image into 784 features.

- * Design Neural Network :

 - Input layer.

 - Hidden layer.

 - Output layer.

- * Compile Model :

 - loss function = categorical Cross-Entropy

 - optimizer = Adam.

 - Metric = Accuracy.

- * Fit the network on training data for given epochs.

- * ~~Test~~ the data. and calculate accuracy.

- * ~~Use~~ the model to classify new handwritten digit images

- * stop.

PSEUDOCODE :

BEGIN

IMPORT necessary libraries

LOAD MNIST dataset

NORMALIZE dataset (divide pixel values by 255)

FLATTEN image into 1D arrays.

ONE-HOT ENCODE labels.

INITIALIZE model as Sequential.

ADD Dense layer (128 neurons, ReLU activation,
input = 784)

ADD Dense layer (10 neurons, Softmax activation)

COMPILE model (optimizer = adam, loss = categorical
cross entropy, metrics = accuracy)

TRAIN model on training data for given epochs

EVALUATE model on test data

DISPLAY accuracy.

END

OBJECTIVE:

→ To load and preprocess the MNIST dataset for neural network input

→ To build feed forward NN to hidden layers.

→ To train model using s-gradient descent.

→ Evaluate the trained model

→ Predict the class

OBSERVATION:

→ The loss decreases with each, showing that the model is learning.

→ Accuracy improves steadily during training.

Epoch	Accuracy	Loss
1	0.9929	0.0232
2	0.9968	0.0128
3	0.9976	0.0099
4	0.9976	0.0088
5	0.9987	0.0058

overall accuracy of the model on the entire database = 0.9828

RESULT:

Therefore the experiment is successfully completed.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                   validation_data=(x_test, y_test),
                   epochs=5,
                   batch_size=128)

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_acc*100:.2f}%")

import matplotlib.pyplot as plt
import numpy as np

index = 10
plt.imshow(x_test[index], cmap='gray')
plt.title("Actual Label: " + str(np.argmax(y_test[index])))
plt.show()

prediction = model.predict(x_test[index].reshape(1,28,28))
print("Predicted Label:", np.argmax(prediction))
```

2025-08-22 09:31:13.717594: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 3s 0us/step

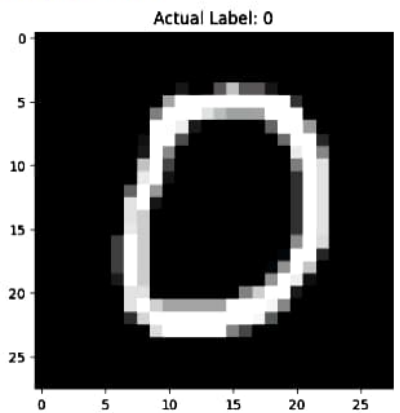
/home/jupyter-r2311847010008/.local/lib/python3.10/site-packages/keras/src/layers/resizing/flattening.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input` object as the first layer in the model instead.

super().__init__(**kwargs)

2025-08-22 09:31:33.970982: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

Epoch 1/5
469/469 ————— 4s 6ms/step - accuracy: 0.9031 - loss: 0.3526 - val_accuracy: 0.9431 - val_loss: 0.1908
Epoch 2/5
469/469 ————— 3s 5ms/step - accuracy: 0.9540 - loss: 0.1610 - val_accuracy: 0.9619 - val_loss: 0.1320
Epoch 3/5
469/469 ————— 3s 5ms/step - accuracy: 0.9658 - loss: 0.1159 - val_accuracy: 0.9678 - val_loss: 0.1144
Epoch 4/5
469/469 ————— 3s 5ms/step - accuracy: 0.9743 - loss: 0.0890 - val_accuracy: 0.9729 - val_loss: 0.0956
Epoch 5/5
469/469 ————— 2s 5ms/step - accuracy: 0.9793 - loss: 0.0721 - val_accuracy: 0.9745 - val_loss: 0.0870

Test Accuracy: 97.45%



1/1 ————— 0s 81ms/step
Predicted Label: 0